

Benchmarking Classification Methods

Matthew Clifford, Fatima Daher, Clayton Durst, Noah Greco, Syed Ahmed Musavi, Rohit Vempati

Abstract

This study examines and compares the classification performance of rpart, support vector machine (SVM), logistic regression, linear discriminant analysis (LDA), naïve Bayes, and random forest models using data sourced from the Comprehensive R Archive Network (CRAN). The analysis encompasses 20 datasets containing a combination of numerical and categorical variables. Averages of accuracy, AUC, and computation types are compared for each method across identical training and testing sets to obtain a baseline performance metric. For LDA and SVM, hyperparameter tuning is implemented and new metrics are obtained. We find that random forest and SVM are generally strong methods in terms of accuracy and AUC, but no single method is the strongest for all datasets. We also find SVM to be the slowest of all methods and naïve Bayes to be the fastest. This leads us to conclude that when deciding upon a method to utilize, accuracy and computation time must be balanced against the needs of the specific project.

Background

In this research study, we aim to conduct a comprehensive benchmarking analysis of various classification methods available within the Comprehensive R Archive Network (CRAN). The study is designed to compare the performance of at least six different methods for classification tasks, leveraging publicly acquired datasets. These methods could include algorithms like logistic regression, random forests, rpart, support vector machines, naïve Bayes, and linear discriminant analysis. And while quadratic discriminant analysis was looked at it wasn't used in the measure comparison like the others. Through various public domains, we were able to acquire 20 different datasets, which we named the following:

Beans, billionaires, cancer, car, diabetes, employee, fires, glass, Haberman, heart, home_loan, iris, maternal, mobile, penguins, pokémon, princess, student, synthetic, titanic.

Each dataset was carefully preprocessed to meet the requirements of the binary classification. This process includes cleaning, handling missing values, and target variable recording.

The challenge lies in comparing the performance of six different methods across diverse datasets, considering aspects such as test set accuracy, AUC, and computation time. The study aims to provide a thorough assessment of these methods, contributing to the understanding of their strengths and weaknesses. Another challenge faced was.

The significance of this study lies in its potential to guide practitioners and researchers in selecting appropriate methods for their specific tasks. By systematically comparing methods on multiple datasets and performance metrics, the study contributes to the broader discourse on the effectiveness of classification techniques.

The primary goal is to benchmark and compare the performance of diverse classification methods available in CRAN. The study aims to go beyond a mere comparison by ensuring statistical validity through multiple runs and various metrics, providing a nuanced understanding of method performance.

This study is significant as it addresses a practical need in machine learning and statistical analysis, providing clarity on the performance of various methods. It is interesting because it explores the nuances of method performance on different datasets, considering real-world complexities. The relevance stems from the increasing reliance on statistical models in decision making, making it crucial to understand the tradeoffs between methods. Investigating this topic is worth the effort, as it directly impacts the choices made by practitioners in selecting the most suitable classification or regression method for their applications.

Methodology

The logistic regression model uses a logit function to help determine the relationship between variables.¹ The model in our case is being used for binary classification, where the dependent variable is categorical and has two possible outcomes (e.g., 0 or 1, True or False, Yes or No). It examines the relationship between one or more independent variables and the probability of a particular outcome.¹ What makes it so great is that it is easy to use, can be used for multiple classes if needed, and makes no assumptions about distributions of classes in feature space. The disadvantages are that is the assumption of linearity between the dependent variable and the independent variables.²

Support Vector Machine (SVM) tries to optimize the space of the margin between classes, but also seeks to constrain the extent of this margin in relation to data points.³ They can be used for both classification and regression models and are classified as supervised learning models.³ SVM is great when there is an understandable margin of dissociation between classes, in high-dimensional spaces, and where the number of dimensions is larger than the number of specimens.⁴ The disadvantage to SVM is that it isn't the best for large data sets, and it does not execute very well when the data set has more sound.⁴

Linear Discriminant Analysis (LDA) is like logistic regression in that both create a linear decision boundary. LDA does this by computing "discriminant scores" for each observation to classify what response variable class it is in. These scores are obtained by finding linear combinations of the independent variables. Quadratic Discriminant Analysis (QDA) is like LDA, but it has key differences in that QDA does not assume constant covariance matrix across classes, while LDA does, and QDA is much better for non-linear cases because it provides a non-linear quadratic decision boundary.⁵ Because this study looked at LDA more in depth, we will discuss its limitations. LDA is effective in pattern recognition and data mining. However, it has the disadvantages in that it is sensitive to outliers, and it assumes a Gaussian distribution, which might not be case.⁶

Decision trees is a model used to handle datasets that tend to be non-linear and are divided into categorical variable and continuous variable decision trees. Categorical variable decision trees will divide the categorical target variables into categories (yes or no).⁷ While a continuous variable decision tree is a decision tree with a continuous target variable (income based on continuous variables). The advantage of decision trees is that they are easy to read and easy to prepare.⁷ The disadvantage is that small changes in the data can result in a major shift in the structure of the decision tree and they are not as effective as predicting the outcome of continuous variables.⁷ While rpart and random forest are both models that use decision trees, the way they approach their models is different. Rpart and random forest are used for classification and regression trees. Rpart draws a tree where the tree is split into different branches by variables and to predict the outcome you must follow the splits and predict the most frequent outcomes.⁸ Random Forest builds multiple decision trees, then compiles the results from all the decision trees which eventually leads to the outcome.⁸

Naïve Bayes is a classification technique based on Bayes' Theorem with an independence assumption among predictors.⁹ The naive Bayes classifier assumes that all features in the input data are normally distributed and independent of each other.⁹ It also assumes that all features contribute equally to the outcome.¹⁰ What is good about Naïve bayes is that it is a simpler classifier compared to other models and it scales well. The disadvantage of naïve bayes is that it an unrealistic assumption that cannot be found in the real world.¹⁰

Originally, our plan encompassed testing the Quadratic Discriminant Analysis (QDA) alongside SVM, logistic regression, LDA, naïve Bayes, and random forest. However, we encountered an error due to excessively

correlated variables when implementing the QDA code. Consequently, we opted to pivot and utilize the “rpart” approach instead.

Proceeding with the application of models and datasets involved:

1. Select a dataset.
2. Split the data into 70% training and 30% validation.
3. For each method, build a model on the training set.
4. Calculate the training time, validation set accuracy, and AUC for each model.
5. Repeat steps 2 – 4 a total of 20 times.
6. Calculate the average of the 20 metrics for each method.
7. Store the method averages for the given dataset
8. Go back to step 1.

Below is a snippet of code that shows how the metrics are generated. Although this code is for logistic regression, the other models operate similarly. This bit of code is wrapped within loops to achieve the desired number of trials for each dataset.

```
## Logistic Regression ##

# Start timer
start <- proc.time()

# Run logistic regression model
md <- stats::glm(as.formula(equation),
                 data = df, subset = train, family = binomial)

# End timer and store the elapsed time
end <- proc.time() - start
times[j, 'log_reg'] <- end['elapsed']

# Make predictions on the validation set
pred_probs <- predict(md, df[valid, ], type = 'response')
pred <- ifelse(pred_probs > 0.5, 1, 0)
true <- df[valid, ncol(df)]
valid_acc[j, 'log_reg'] <- mean(pred == true)

# Get the AUC
ROC_df <- data.frame('true_labels' = true, 'probs' = pred_probs)
AUCs[j, 'log_reg'] <- ROC_func(ROC_df, 1, 2)
```

Next, we fine-tuned two models: Support Vector Machine and Linear Discriminant Analysis. The support vector classifier involves a function called a kernel. The kernel type affects the decision boundary, and each one has unique parameters. We test the following:

- Linear kernel with no parameters
- Polynomial kernel with different values of the degree parameter
- Radial kernel with different values of the gamma parameter

We also test different values of the cost parameter for each kernel. The cost parameter adjusts how many training observations can be misclassified. A model with a high cost might fit the training data very well but behave poorly when given new data. The parameter tuning was done using the `e1071::tune` function.

```
# Loop through each kernel type
for (x in kernels) {

  # For a linear kernel ...
  if (x == 'linear') {

    # Test different costs using 10-fold cross validation
    tuning1 <- e1071::tune(e1071::svm, as.formula(equation), data = df[train, ],
                          kernel = 'linear',
                          ranges = list(cost = costs))

  } else if (x == 'polynomial') {

    # Test different cost-degree combinations using 10-fold cross validation
    tuning2 <- e1071::tune(e1071::svm, as.formula(equation), data = df[train, ],
                          kernel = 'polynomial', ranges = list(cost = costs,
                                                                degree = degrees))

  } else {

    # Test different cost-gamma combinations using 10-fold cross validation
    tuning3 <- e1071::tune(e1071::svm, as.formula(equation), data = df[train, ],
                          kernel = 'radial', ranges = list(cost = costs,
                                                            gamma = gammas))

  }

}
```

Shrinkage is a regularization technique in Linear Discriminant Analysis (LDA), crucial for improving model performance when dealing with datasets that have a high feature-to-observation ratio or correlated features. It adjusts the variance-covariance matrix estimates, balancing between reducing variance and adding some bias, thus aiding in preventing overfitting and enhancing model robustness.

In our model tuning, shrinkage was the key hyperparameter. Developed a custom R function, ‘`tune_lda`’, to automate its tuning across various datasets. This function explored shrinkage values from 0 to 1 in 0.1 increments, applying a 70-30 split for training and testing in an iterative manner, repeated 20 times per dataset for reliability. Model performance was evaluated based on accuracy, and the optimal shrinkage value for each dataset was determined as the one yielding the highest average accuracy. This systematic approach ensured a thorough and efficient optimization of the shrinkage parameter in LDA models.

```
# Define LDA hyperparameter tuning function
tune_lda <- function(data, formula, response_col, shrinkage_values) {
  accuracy_values <- numeric(length(shrinkage_values))
  for (shrinkage in shrinkage_values) {
    train_indexes <- matrix(ncol = 20, nrow = 0.7 * nrow(data))
    test_indexes <- matrix(ncol = 20, nrow = nrow(data) - nrow(train_indexes))
    for (i in 1:20) {
      train_index <- sample(1:nrow(data), 0.7 * nrow(data))
      train_indexes[, i] <- train_index
    }
  }
}
```

```

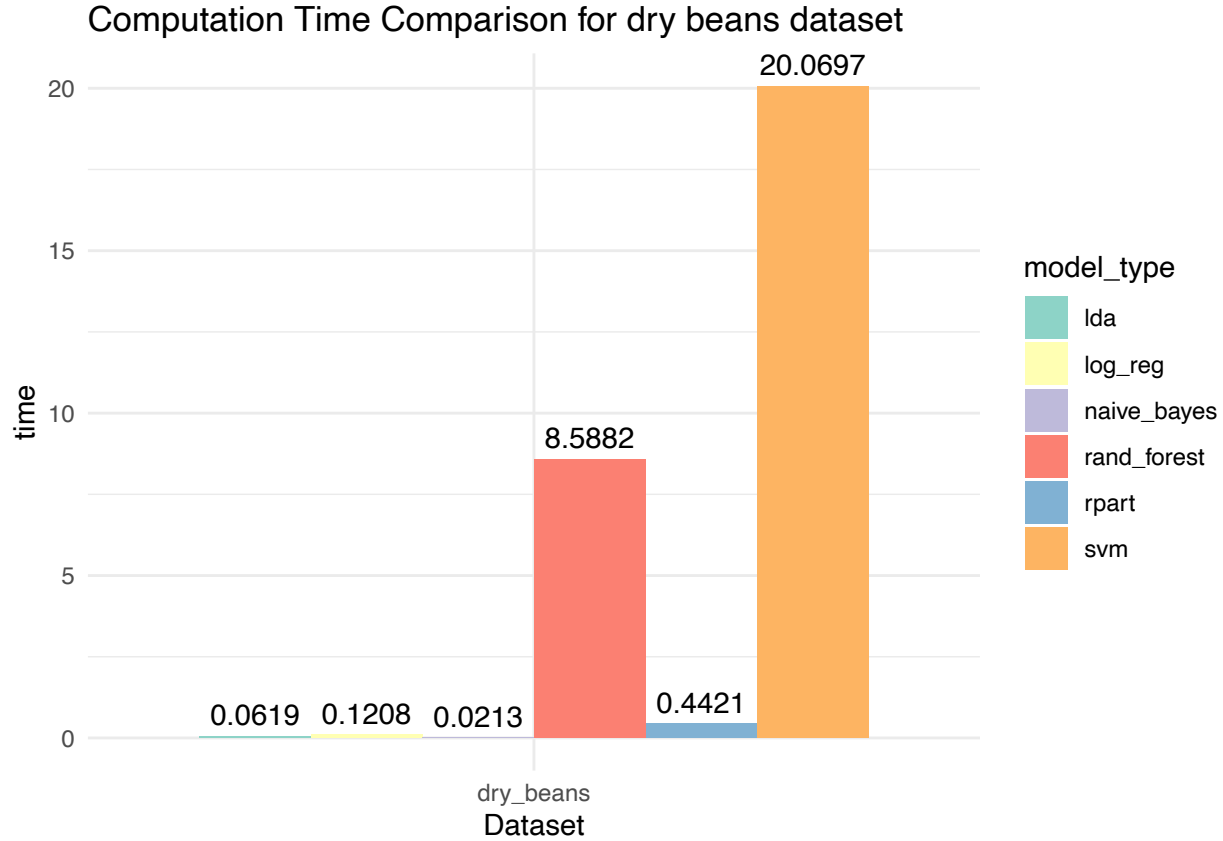
test_index <- setdiff(1:nrow(data), train_index)
test_indexes[, i] <- test_index
}
lda_model <- MASS::lda(formula, data = data[train_indexes[, 1], ],
  shrinkage = shrinkage)
lda_prediction <- predict(lda_model, data[test_indexes[, 1], ], type = "class")
accuracy_values[which(shrinkage_values == shrinkage)] <-
  sum(lda_prediction$class == data[test_indexes[, 1], response_col]) /
  nrow(data[test_indexes[, 1], ])
}
best_shrinkage <- shrinkage_values[which.max(accuracy_values)]
return(list(best_shrinkage = best_shrinkage, accuracy_values = accuracy_values))
}

```

Results

Table 1: Average Training Time (s)

dataset	n	p	log_reg	naive_bayes	lda	rpart	rand_forest	svm
billionaires	2407	15	0.01485	0.00605	0.00945	0.05295	0.87820	0.53275
breast_cancer	569	30	0.02510	0.01185	0.00860	0.02115	0.20665	0.02685
car_details	1874	12	0.01215	0.00570	0.00705	0.02310	0.44085	0.11045
diabetes	709	6	0.00315	0.00270	0.00305	0.00940	0.17040	0.03110
dry_beans	13611	16	0.12080	0.02130	0.06190	0.44205	8.58820	20.06965
employee	4653	8	0.01490	0.00435	0.01425	0.04455	0.94375	2.04650
fires	244	13	0.00940	0.00510	0.00395	0.00590	0.04330	0.00815
glass	214	9	0.00435	0.00330	0.00300	0.00485	0.04225	0.00610
haberman	305	3	0.00200	0.00130	0.00210	0.00505	0.05250	0.00855
heart	303	10	0.00430	0.00260	0.00350	0.00720	0.07650	0.01435
home_loan	566	9	0.00445	0.00265	0.00390	0.00915	0.15525	0.04025
iris	1200	20	0.02945	0.00855	0.00885	0.02760	0.33420	0.05185
maternal_health	1014	6	0.00405	0.00235	0.00305	0.00870	0.18890	0.03595
mobile	2000	20	0.03695	0.01200	0.01105	0.03035	0.79860	0.31195
penguins	3430	10	0.01580	0.00460	0.01210	0.05755	1.10325	1.59845
pokemon	684	33	0.03785	0.00990	0.01095	0.01065	0.14610	0.02835
princess	2338	13	0.00975	0.00530	0.00750	0.04945	0.93300	0.33190
student	9873	5	0.03790	0.00580	0.01135	0.03830	1.92720	1.35980
synthetic	800	2	0.00365	0.00145	0.00195	0.00500	0.12245	0.02140
titanic	712	7	0.00430	0.00230	0.00345	0.00915	0.15240	0.04150

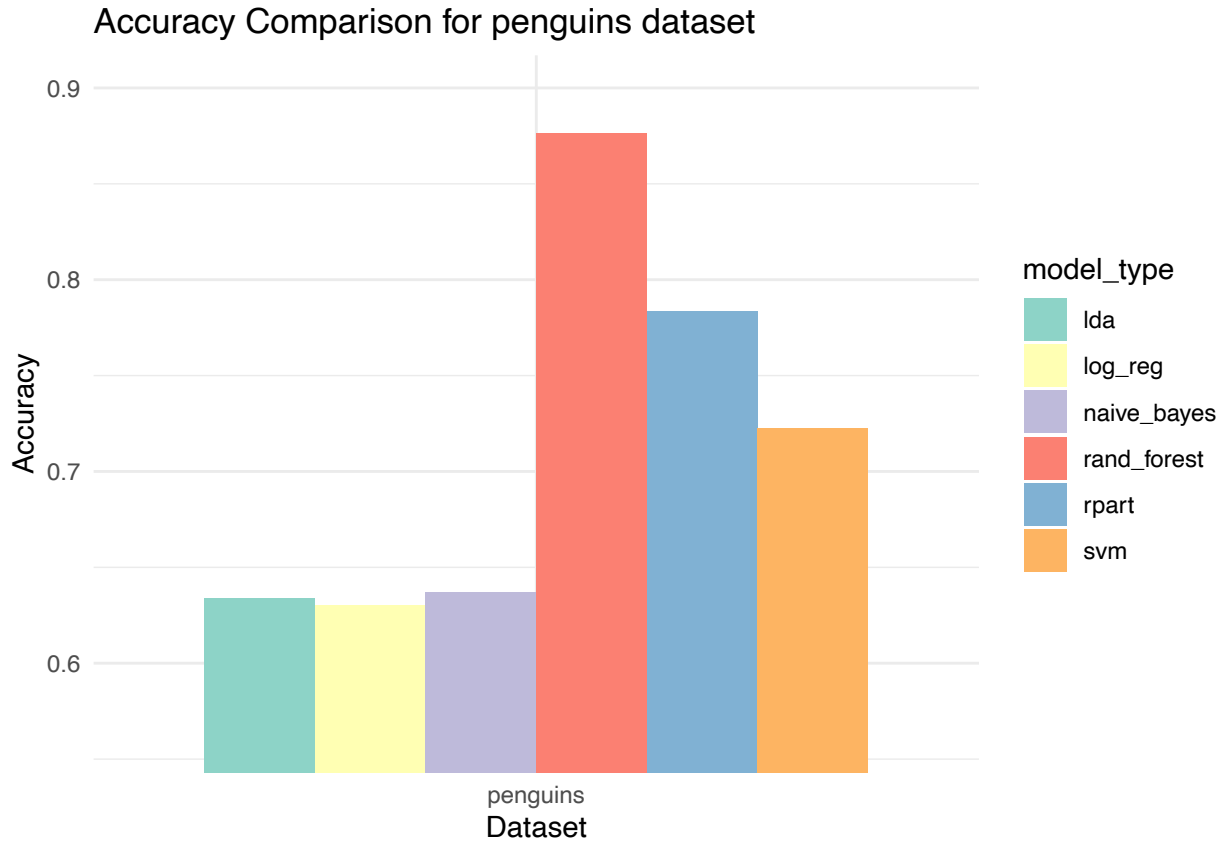


This dataset highlights a potential downside of using SVM and random forests. The SVM model took over 20 seconds to execute. This was by far the largest outlier for this metric and shows that SVM as well as random forest models can struggle on large datasets.

Table 2: Average Validation Set Accuracies

dataset	n	p	log_reg	naive_bayes	lda	rpart	rand_forest	svm
billionaires	2407	15	0.7576763	0.6876902	0.7577455	0.7756570	0.7756570	0.7812586
breast_cancer	569	30	0.9426901	0.9327485	0.9578947	0.9263158	0.9593567	0.9774854
car_details	1874	12	0.9396092	0.8698046	0.9189165	0.9284192	0.9561279	0.9478686
diabetes	709	6	0.7659624	0.7607981	0.7664319	0.7413146	0.7680751	0.7572770
dry_beans	13611	16	0.7952008	0.7516283	0.7967679	0.7962047	0.7742287	0.7961680
employee	4653	8	0.7340974	0.7307307	0.7328080	0.8283668	0.8579513	0.8252507
fires	244	13	0.9351351	0.9547297	0.9175676	0.9709459	0.9783784	0.9439189
glass	214	9	0.9315385	0.9130769	0.9253846	0.9238462	0.9423077	0.9361538
haberman	305	3	0.7538043	0.7570652	0.7543478	0.7239130	0.7222826	0.7423913
heart	303	10	0.8104396	0.8274725	0.8203297	0.7719780	0.8109890	0.7912088
home_loan	566	9	0.7411765	0.6967647	0.7464706	0.7038235	0.7291176	0.7491176
iris	1200	20	0.9809722	0.9450000	0.9805556	0.9741667	0.9861111	0.9926389
maternal_health	1014	6	0.8411475	0.8722951	0.8563934	0.9000000	0.9424590	0.9054098
mobile	2000	20	0.9864167	0.9375833	0.9625000	0.9411667	0.9508333	0.9590000
penguins	3430	10	0.6300777	0.6370262	0.6337707	0.7833333	0.8762391	0.7223518
pokemon	684	33	0.9975728	0.9834951	0.9983010	0.9907767	0.9973301	0.9953883
princess	2338	13	0.8727208	0.7969373	0.8676638	0.8255698	0.9295584	0.9558405
student	9873	5	0.9722316	0.9203241	0.9578832	0.9561107	0.9663572	0.9703241
synthetic	800	2	0.8735417	0.8658333	0.8645833	0.8547917	0.8504167	0.8739583
titanic	712	7	0.8004673	0.7864486	0.7946262	0.8009346	0.8165888	0.8189252

dataset	n	p	log_reg	naive_bayes	lda	rpart	rand_forest	svm
---------	---	---	---------	-------------	-----	-------	-------------	-----

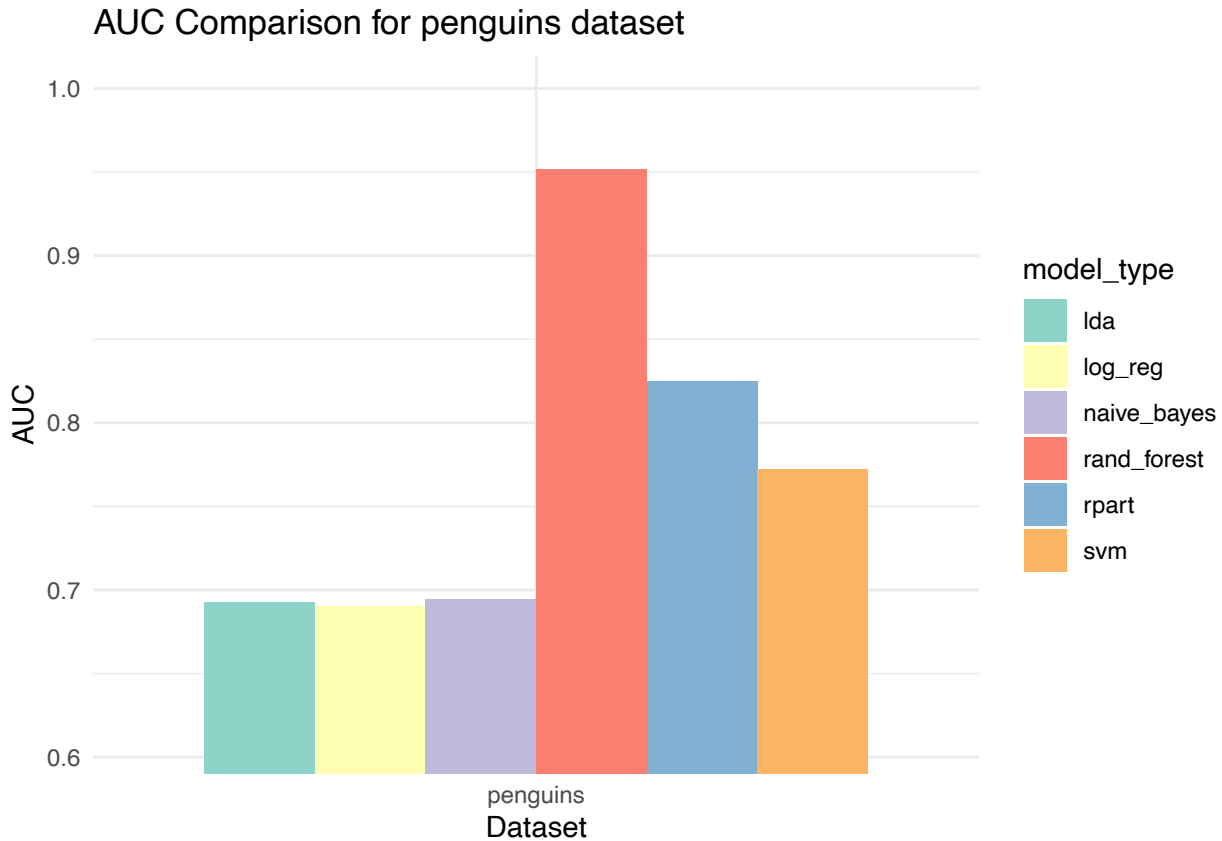


The penguins dataset highlights a general trend in regard to model accuracy. There was a lot of variation in the results of each model. We can see random forest, rpart and support vector machine models far outcompete linear discriminant analysis and naive Bayes.

Table 3: Average AUC Values

dataset	n	p	log_reg	naive_bayes	lda	rpart	rand_forest	svm
billionaires	2407	15	0.7824492	0.7311112	0.7825095	0.7138348	0.8084461	0.8002764
breast_cancer	569	30	0.9673618	0.9846187	0.9918013	0.9589780	0.9917566	0.9963035
car_details	1874	12	0.9845421	0.9531574	0.9763081	0.9463945	0.9914109	0.9901876
diabetes	709	6	0.8337181	0.8233446	0.8337507	0.7776746	0.8238917	0.8210402
dry_beans	13611	16	0.8900454	0.8583013	0.8907872	0.8343786	0.8786106	0.8816543
employee	4653	8	0.7439575	0.7412180	0.7441334	0.8471134	0.8561090	0.8318871
fires	244	13	0.9620325	0.9848534	0.9768670	0.9581487	0.9982217	0.9908351
glass	214	9	0.9767137	0.9544511	0.9708342	0.9890257	0.9886532	0.9853846
haberman	305	3	0.6822700	0.6542739	0.6863776	0.6268626	0.6893456	0.6945711
heart	303	10	0.8956964	0.8917580	0.8996855	0.8254176	0.8906193	0.8873725
home_loan	566	9	0.7179447	0.6958028	0.7193815	0.6450332	0.7172713	0.7010406
iris	1200	20	0.9912038	0.9887539	0.9991297	0.9939677	0.9990823	0.9998805
maternal_health	1014	6	0.9115061	0.8799188	0.9187740	0.9014355	0.9761254	0.9345557
mobile	2000	20	0.9913639	0.9876091	0.9957713	0.9663538	0.9934722	0.9952539
penguins	3430	10	0.6905171	0.6946686	0.6927450	0.8247873	0.9515436	0.7724855
pokemon	684	33	0.6360020	0.5982507	0.6117389	0.4014681	0.7500000	0.7477072

dataset	n	p	log_reg	naive_bayes	lda	rpart	rand_forest	svm
princess	2338	13	0.9351599	0.8743052	0.9333701	0.8661191	0.9758975	0.9865527
student	9873	5	0.9976787	0.9818414	0.9947654	0.9736748	0.9964077	0.9973697
synthetic	800	2	0.9329548	0.9325305	0.9332112	0.8862584	0.9176162	0.9326502
titanic	712	7	0.8516957	0.8267638	0.8514259	0.8243939	0.8656499	0.8432475

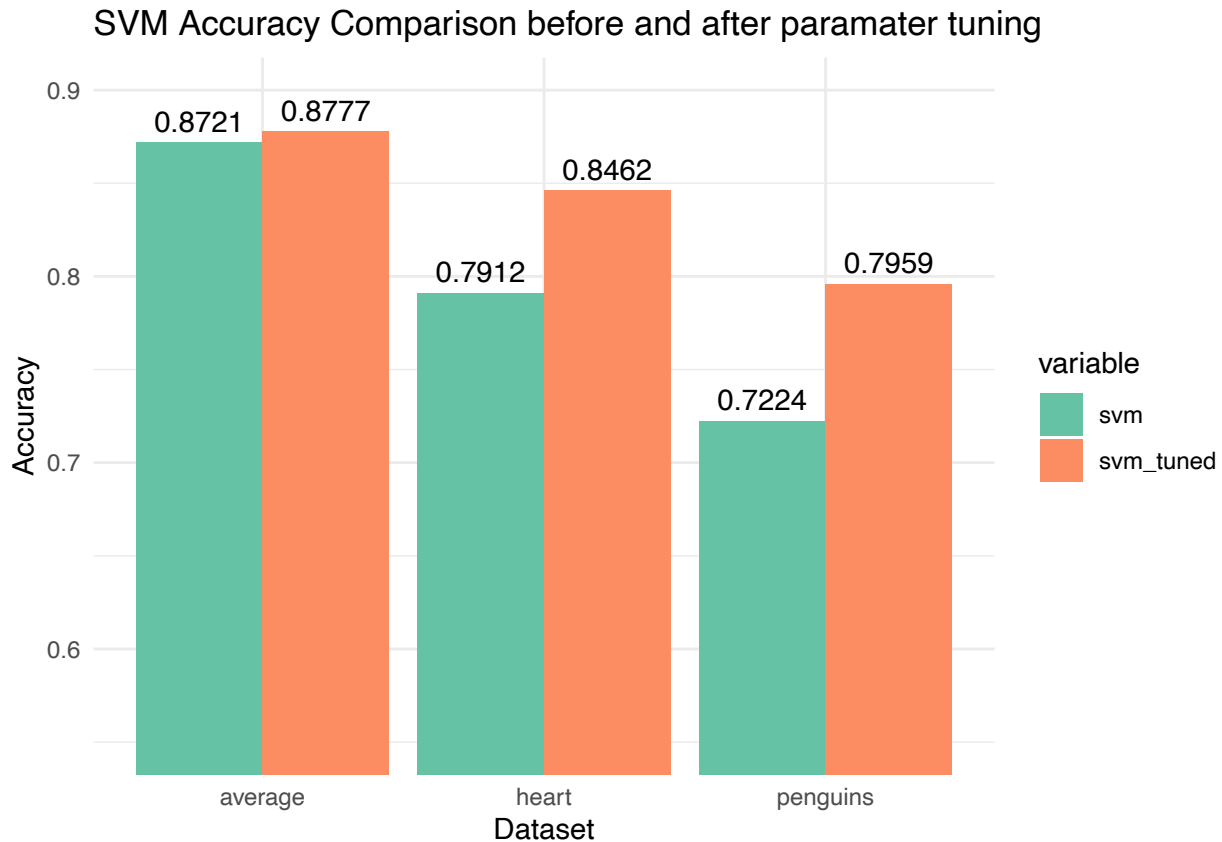


Zooming into the results for the penguins data set again, we can see similar variation in the results for AUC values. This variation applies to the majority of the other datasets.

Table 4: SVM Tuning Results

dataset	svm	svm_tuned	kernel	cost	degree	gamma
billionaires	0.7812586	0.7662517	polynomial	5.0	3	NA
breast_cancer	0.9774854	0.9590643	linear	1.0	NA	NA
car_details	0.9478686	0.9626998	radial	10.0	NA	0.1
diabetes	0.7572770	0.7323944	linear	0.1	NA	NA
dry_beans	0.7961680	0.8031342	linear	0.1	NA	NA
employee	0.8252507	0.8474212	radial	10.0	NA	0.1
fires	0.9439189	0.9594595	linear	5.0	NA	NA
glass	0.9361538	0.9538462	linear	0.1	NA	NA
haberman	0.7423913	0.7282609	polynomial	5.0	5	NA
heart	0.7912088	0.8461538	radial	1.0	NA	0.1
home_loan	0.7491176	0.7411765	radial	1.0	NA	0.1
iris	0.9926389	0.9972222	linear	1.0	NA	NA
maternal_health	0.9054098	0.9081967	radial	10.0	NA	0.1

dataset	svm	svm_tuned	kernel	cost	degree	gamma
mobile	0.9590000	0.9783333	linear	10.0	NA	NA
penguins	0.7223518	0.7959184	radial	10.0	NA	0.1
pokemon	0.9953883	0.9951456	linear	0.1	NA	NA
princess	0.9558405	0.9529915	radial	1.0	NA	0.1
student	0.9703241	0.9696151	linear	1.0	NA	NA
synthetic	0.8739583	0.8583333	radial	5.0	NA	0.1
titanic	0.8189252	0.7990654	polynomial	5.0	3	NA

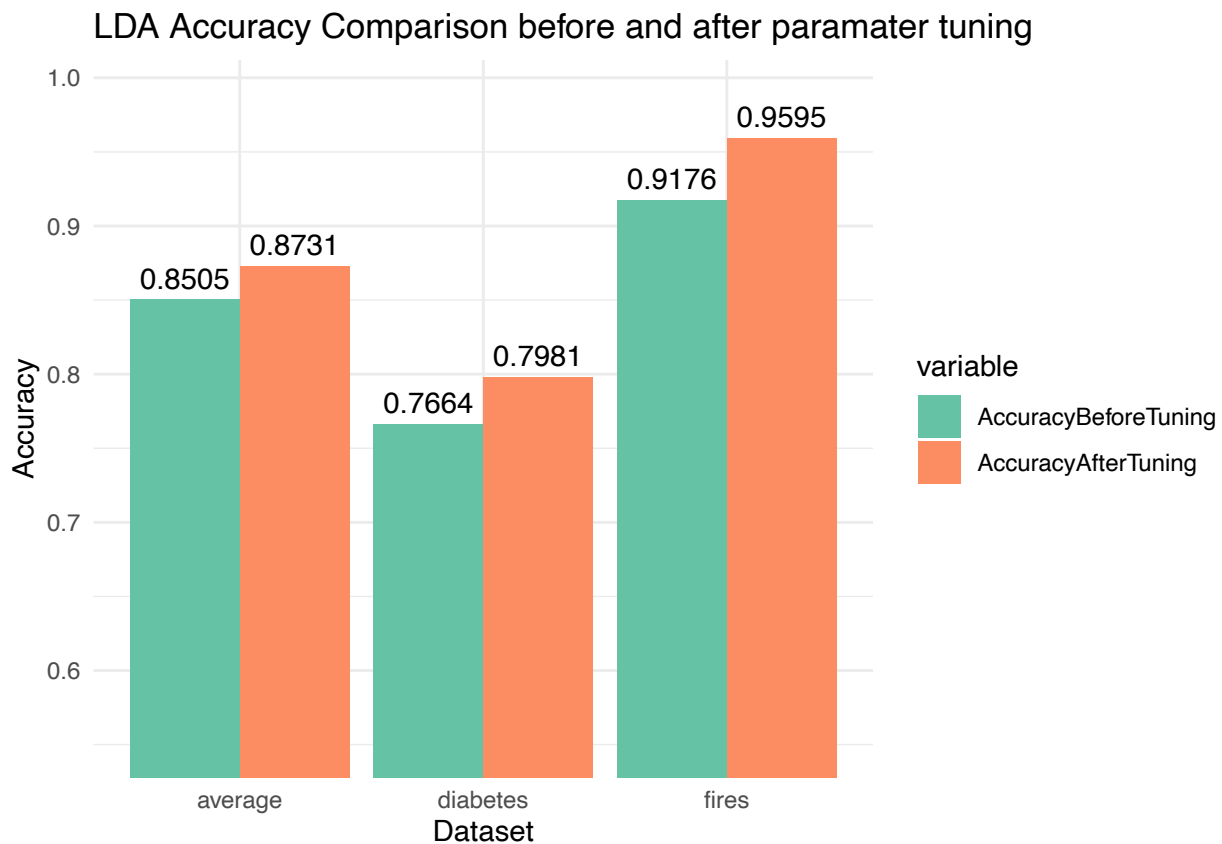


Here we can see the results in the change of accuracy values before and after parameter tuning for the SVM models. Though on average our parameter tuning method resulted in only a modest increase in accuracy, certain data sets (heart and penguins) resulted in a significant increase.

Table 5: LDA Tuning Results

Dataset	AccuracyBeforeTuning	BestShrinkage	AccuracyAfterTuning
billionaires	0.7577455	0.0	0.7731674
breast_cancer	0.9578947	0.9	0.9766082
car_details	0.9189165	0.7	0.9467140
diabetes	0.7664319	0.0	0.7981221
dry_beans	0.7967679	0.3	0.8046033
employee	0.7328080	1.0	0.7392550
fires	0.9175676	0.1	0.9594595
glass	0.9253846	0.3	0.9538462
haberman	0.7543478	0.0	0.7608696

Dataset	AccuracyBeforeTuning	BestShrinkage	AccuracyAfterTuning
heart	0.8203297	0.1	0.8681319
home_loan	0.7464706	0.9	0.8000000
iris	0.9805556	1.0	0.9944444
maternal_health	0.8563934	0.2	0.8786885
mobile	0.9625000	1.0	0.9716667
penguins	0.6337707	0.3	0.6627794
pokemon	0.9983010	0.1	1.0000000
princess	0.8676638	0.9	0.8860399
student	0.9578832	1.0	0.9628629
synthetic	0.8645833	0.0	0.9166667
titanic	0.7946262	0.4	0.8084112



Similarly, parameter tuning for LDA models resulted in a modest increase (0.02) in accuracy on average. However, the diabetes and fires dataset had a much larger increase.

Conclusion

In our observations, generally, no singular or specific model type consistently outperforms others across all scenarios. Hence, choosing a suitable model type should entail an experimental process, involving the testing of each new dataset across various model types, rather than adopting a universal “one-size-fits-all” approach.

This can be observed in the fact that there was a large amount of variation in the results of our performance metrics and parameter tuning for different model types across the twenty datasets. When looking at the performance metrics of accuracy, AUC values, and computation time we see each model type performing

at varying degrees. Support Vector Machines and Random Forest had the highest accuracy in seven of our data sets and linear discriminant analysis, logistic regression, and naive Bayes each had the highest accuracy across two data sets. Results for AUC values were similarly spread out across the model types. Random forest, linear discriminant analysis, and support vector machines performed best for eight, six, and four data sets respectively. Finally, naive Bayes outperformed all models by far in computation time, performing best for 16 of the datasets.

We noticed that the random forest algorithm excelled consistently across various datasets, demonstrating strong performance in accuracy and AUC values across most of the datasets. Additionally, it managed computational tasks efficiently even with larger datasets, without significant delays. Another model that excelled was SVM, which did well in accuracy and AUC values, having the third highest AUC values and being tied with random forest for most datasets with the highest accuracies at seven. The tradeoff for both random forest and SVM is that they both come with notably large penalties in computation time. Alternatively, naïve Bayes computation time was the best of all our model types, however, it performed poorly relative to the other models used in terms of accuracy and AUC. We assume this discrepancy occurs due to the nature of naïve Bayes and the oversimplified assumptions it makes about the data.

We believe that the shape of the decision boundary in each dataset plays a significant role in which model performs best. We further speculate that when the decision boundary is linear, methods like logistic regression will be able to compete with more complex methods like random forests and support vector machines. This is partially why support vector machines are so versatile. Since they can have linear, polynomial, or radial kernels, they can be applied to a wider variety of datasets with different kinds of decision boundaries.

One problem we encountered was extremely strong relationships between the dependent and independent variables within certain datasets. Consequently, our models achieved high accuracies relatively easily due to the substantial relationship between the variables. For future experiments, we might opt for datasets where the relationship between the dependent and independent variables is not as obvious, as running such models might not yield meaningful insights or improvements.

A noteworthy observation arose in SVM when performing hyperparameter tuning. Contrary to the expected improvements, certain accuracies decreased. This surprising outcome was possibly due to employing a generalized method not geared toward a specific dataset. It is hypothesized that if the models were tuned individually for each dataset rather than using a broad approach, the accuracies might have shown an increase. Unfortunately, SVM hyperparameter tuning is computationally expensive and so we adopted a general approach. This phenomenon did not occur in LDA during hyperparameter tuning, and it demonstrates a need for research on why shrinkage performed better during hyperparameter tuning.

An important takeaway was the necessity of aligning the data with the intended model for learning purposes. This lesson surfaced repeatedly and prompted our shift from utilizing QDA to rpart. QDA encountered issues with specific datasets, causing crashes and hindering the progression of the loop executing other codes. We would have done QDA separately but the comparison between QDA and other models would not have been fair because it would not be under the same circumstances as others and therefore would not have been reliable. Overall, the study showed the need for an adaptable approach in model selection which considers the unique characteristics of each dataset rather than relying on a universally superior model.

References

1. Amazon Web Services. “Logistic Regression.” AWS, Amazon, www.aws.amazon.com/what-is/logistic-regression/#:~:text=Logistic%20regression%20is%20a%20data,outcomes%2C%20like%20yes%20or%20no.
2. GeeksforGeeks. “Advantages and Disadvantages of Logistic Regression.” GeeksforGeeks, www.geeksforgeeks.org/advantages-and-disadvantages-of-logistic-regression/.
3. RStudio. “Classification: Logistic Regression, LDA and SVM.” RStudio Pubs, Amazon Web Services, rstudio-pubs-static.s3.amazonaws.com/460138_312ecffcfde94d5b8f2797f20f39dd2e.html.

4. GeeksforGeeks. “Support Vector Machine in Machine Learning.” GeeksforGeeks, www.geeksforgeeks.org/support-vector-machine-in-machine-learning/.
5. UC Riverside. “Linear Discriminant Analysis (LDA).” UC Riverside, University of California, uc-r.github.io/discriminant_analysis#:~:text=Both%20LDA%20and%20logistic%20non%2Dlinear%20quadratic%20decision%20boundary.
6. Typeset.io. “What Are the Advantages and Disadvantages of Using the LDA?” Typeset, typeset.io/questions/what-are-the-advantages-and-disadvantages-of-using-the-lda-2spww00jyn.
7. Corporate Finance Institute. “Decision Tree - What is it and How Does it Work?” Corporate Finance Institute, www.corporatefinanceinstitute.com/resources/data-science/decision-tree/#:~:text=Compared%20to%20other%20decision%20techniques,having%20to%20compute%20complex%20calculations.
8. Medium. “What is the Difference Between rpart and Random Forest Package?” Medium, Snehi Bhaskar, medium.com/@snehi.bhaskar26/what-is-the-difference-between-rpart-and-random-forest-package-15463fe74026.
9. Analytics Vidhya. “Naive Bayes Explained: What Is the Naive Bayes Algorithm?” Analytics Vidhya, www.analyticsvidhya.com/blog/2017/09/naive-bayes-explained/#What_Is_the_Naive_Bayes_Algorithm?
10. IBM. “Naive Bayes - IBM.” IBM, www.ibm.com/topics/naive-bayes#:~:text=The%20Na%C3%AFve%20Bayes%20classifier%20is,a%20given%20class%20or%20category.
11. James, Gareth, et al. An Introduction to Statistical Learning with Applications in R. 2nd ed., Corrected Printing, June 21, 2023.
12. Provost, Foster, and Tom Fawcett. Data Science for Business: What You Need to Know About Data Mining and Data-Analytic Thinking. First ed., July 2013.